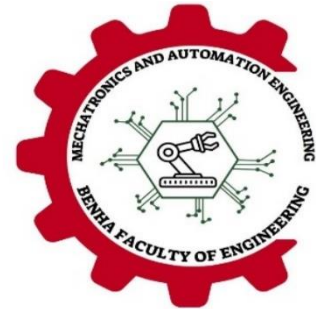




Benha University
Benha Faculty of Engineering
Mechatronics and Automation Engineering Program
MAM 301 Design of Mechatronics
Design project

Student names: Mahmoud Mohamed Shamekh

Omar Mahmoud Metwally

Youssef Mostafa Ayad

Ebrahim Magdy Ebrahim

Supervisor Name: Dr/ Amro Shafik

Eng/ Mohamed Ashraf

Abstract	4
Conceptual Design for Ball and Beam system	4
1. Problem Definition[2]	4
2. System Objectives	5
3. System Needs	5
4. System Specifications	5
5. Constraints	6
5.1 Safety Constraints	6
5.2 Environmental Constraints	6
5.3 Manufacturability Constraints.....	6
8. Design Evaluation Criteria.....	8
9. Selected Conceptual Design.....	9
Preliminary System	9
Mechanical design	9
2.preliminary electronic/control circuit diagram.....	10
3.All components and their interfaces	10
Mechanical Design, Kinematics, and FEA.....	12
System Overview.....	12
2. Geometry & Dimensions	12
Material Selection.....	13
Kinematic Relation	13
Dynamics.....	14
Stress Analysis	15
Summary	15
Actuator Verification.....	15
Finite Element Analysis (FEA).....	16
FEA Summary.....	18
Final Mechanical Model Validation	18

Mathematical Modeling.....	19
Simulink Simulation.....	28
Hardware–Software Integration and Data Visualization	31
Arduino code.....	31
MATLAB Code.....	34
System Identification & Parameter Estimation	36
Comparison Between Simulation and Practical Work	37
Conclusion	41
References.....	42

Symbol	Description	Unit
(x)	Ball position along the beam	m
$(\dot{x})$	Ball linear velocity	m/s
$(\ddot{x})$	Ball linear acceleration	m/s ²
(α)	Beam inclination angle	rad
$(\dot{\alpha})$	Beam angular velocity	rad/s
$(\ddot{\alpha})$	Beam angular acceleration	rad/s ²
(θ)	Servo motor shaft angle	rad
$(\dot{\theta})$	Servo motor angular velocity	rad/s
$(\ddot{\theta})$	Servo motor angular acceleration	rad/s ²
(m)	Mass of the ball	kg
(m_b)	Mass of the beam	kg
(r)	Radius of the ball	m
(L)	Length of the beam	m
(R)	Distance from beam pivot to ball	m

(d)	Lever arm length	m
(J)	Moment of inertia of the ball	$\text{kg}\cdot\text{m}^2$
(J_b)	Moment of inertia of the beam	$\text{kg}\cdot\text{m}^2$
(J_m)	Motor rotor inertia	$\text{kg}\cdot\text{m}^2$
(g)	Gravitational acceleration	m/s^2
(τ)	Applied torque	$\text{N}\cdot\text{m}$
(τ_m)	Motor torque	$\text{N}\cdot\text{m}$
(V_a)	Motor armature voltage	V
(I_a)	Motor armature current	A
(R_a)	Motor armature resistance	Ω
(L_a)	Motor armature inductance	H
(K_t)	Motor torque constant	$\text{N}\cdot\text{m/A}$
(K_e)	Motor back-EMF constant	$\text{V}\cdot\text{s/rad}$
(b)	Equivalent viscous damping coefficient	$\text{N}\cdot\text{s/m}$
(b_m)	Motor viscous damping coefficient	$\text{N}\cdot\text{m}\cdot\text{s/rad}$
(F_f)	Friction force	N
$(P(s))$	Ball and Beam plant transfer function	m/rad
$(C(s))$	PID controller transfer function	—
(K_p)	Proportional gain	—
(K_i)	Integral gain	—
(K_d)	Derivative gain	—
(ω_n)	Natural frequency	rad/s
(ζ)	Damping ratio	—
(e_{ss})	Steady-state error	—

Abstract

This project presents the systematic design, modeling, simulation, and implementation of a Ball and Beam control system following the VDI 2206 V-Model for mechatronic system development. The Ball and Beam system represents a classical unstable control problem that integrates mechanical design, electronics, sensing, and control engineering. The primary objective is to achieve accurate and stable positioning of a rolling ball along a beam by regulating the beam angle using a feedback control strategy.

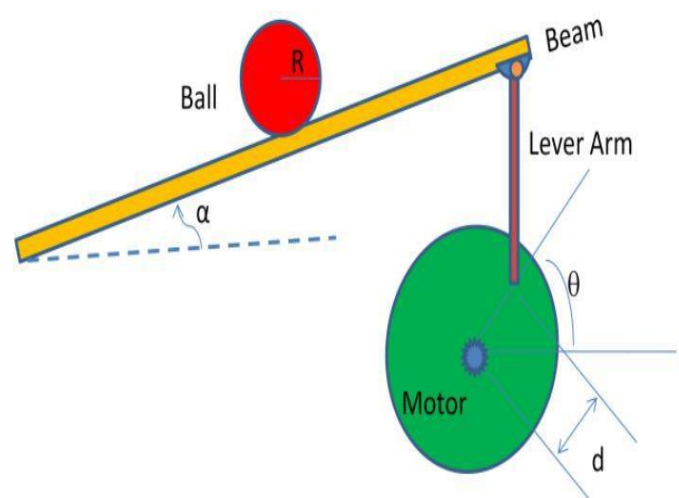
The development process begins with needs analysis and product design specifications, followed by conceptual design and component selection. A mathematical model of the system is derived using Newtonian mechanics, considering the rolling dynamics of the ball and small-angle approximations. Based on the obtained model, a PID controller is designed to stabilize the inherently unstable system and meet performance requirements such as low overshoot, short settling time, and minimal steady-state error. MATLAB/Simulink is used for system simulation and controller tuning, while Hardware-in-the-Loop (HIL) techniques are applied to validate the control strategy before real-time implementation.

Finally, the controller is implemented on an Arduino microcontroller, and experimental results are compared with simulation outcomes. The results demonstrate good agreement between theoretical, simulated, and practical behavior, confirming the effectiveness of the V-Model approach in developing a reliable mechatronic control system.

Conceptual Design for Ball and Beam system

1. Problem Definition^[2]

The objective of the Ball and Beam system is to control the position of a ball along a beam by adjusting the beam's inclination angle using an actuator. The control goal is to balance the ball at a desired position or make it track a reference position with high accuracy.



2. System Objectives

1. Maintain stable balance of the ball on the beam without falling
2. Accurately control the ball position
3. Provide fast response to disturbances and reference changes
4. Allow easy tuning and calibration

3. System Needs

- Continuous measurement of the ball position
- Precise adjustment of the beam angle
- Real-time execution of the control algorithm
- Data visualization and/or logging
- High accuracy and stability
- Reliable operation
- Reasonable cost for educational use
- Electrical and mechanical safety

4. System Specifications

Parameter	Specification	Parameter	Specification
Beam Length (L)	35cm	Controller	PID
Ball Radius (r)	15–20 mm	Power Supply	12 V DC
Ball Mass (m)	50–100 g	microcontroller	Arduino
Beam Angle Range (α)	$\pm 15^\circ$	Overshoot	< 10%
Position Accuracy	± 2 mm	Actuator	Servo Motor MG995
Settling Time	< 3 s	Sensor	Ultrasonic

5. Constraints

5.1 Safety Constraints

- Mechanical end-stops to limit beam rotation
- Electrical insulation and proper grounding
- Overcurrent and overvoltage protection

5.2 Environmental Constraints

- Indoor laboratory operation only
- Operating temperature range: 0–40 °C
- Low vibration and dust-free environment
- No exposure to moisture or corrosive substances

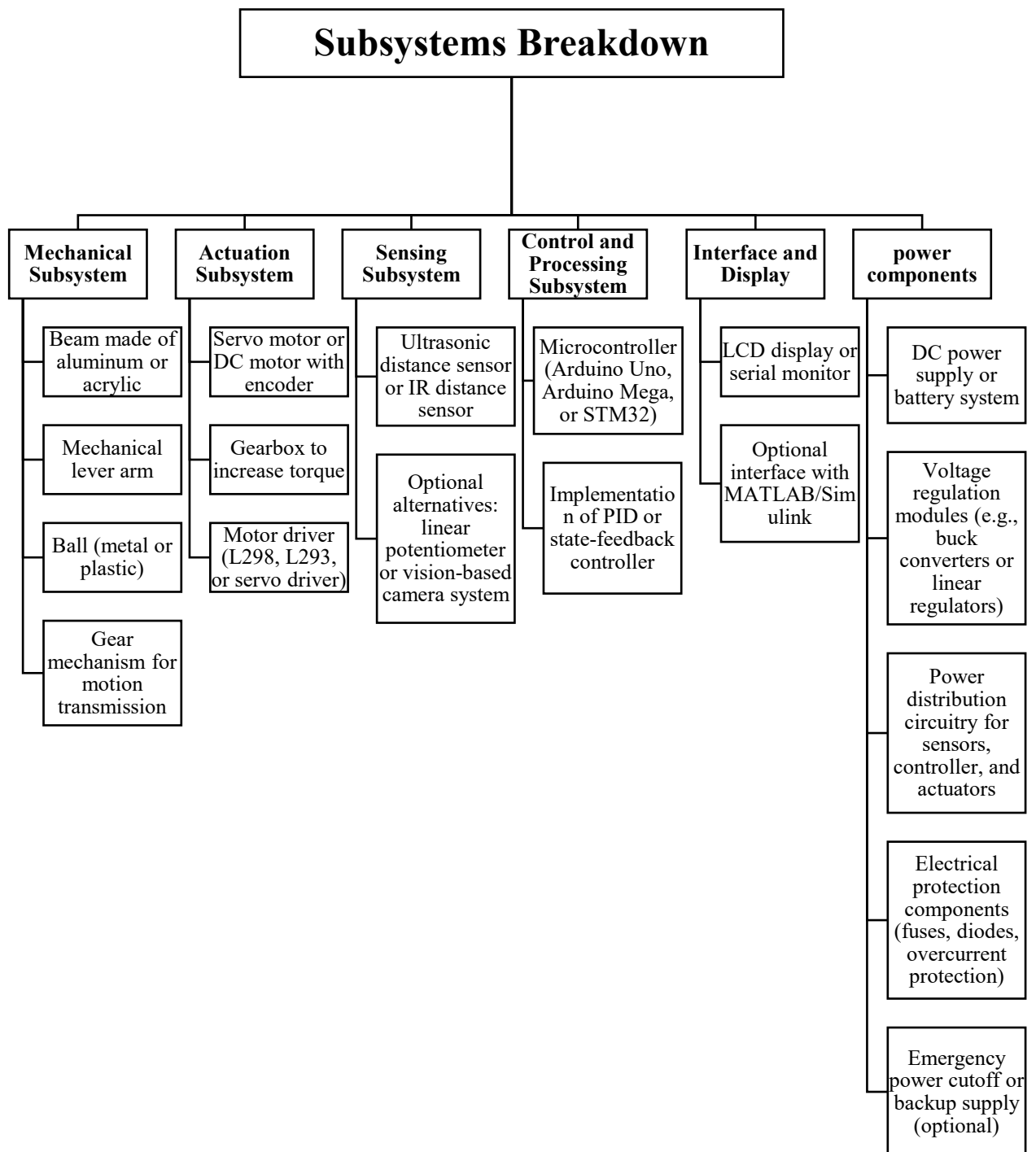
5.3 Manufacturability Constraints

- Use of commercially available components
- Simple machining processes (3D printing)
- Modular design for easy assembly and replacement
- Low manufacturing and maintenance cost



Figure 1: the system

6.



7. Design Alternatives

Design Alternative	DA ₁	DA ₂	DA ₃
Sensor Type	Ultrasonic	IR Sensor	Camera Vision
Actuator	Servo Motor	DC Motor + Encoder	Stepper Motor
Control Method	PID	PI	State-Space
med Controller	Arduino Uno	Arduino Mega	STM32
Cost	Medium	High	High
Complexity	Low	Medium	High

8. Design Evaluation Criteria

Criterion	Weight	DA ₁	DA ₂	DA ₃
Accuracy	0.30	0.7	0.9	0.6
Stability	0.40	0.9	0.7	0.6
Cost	0.10	0.9	0.5	0.6
Ease of Implementation	0.20	0.6	0.8	0.7
evaluation	1	0.78	0.76	0.66

9. Selected Conceptual Design

Selected Design: DA₁

Justification

DA₁ provides the best balance between accuracy, stability, and implementation simplicity. It is well-suited for educational and laboratory environments, offers moderate cost, and allows easy integration with MATLAB and Simulink for control analysis and tuning.

Preliminary System

Mechanical design

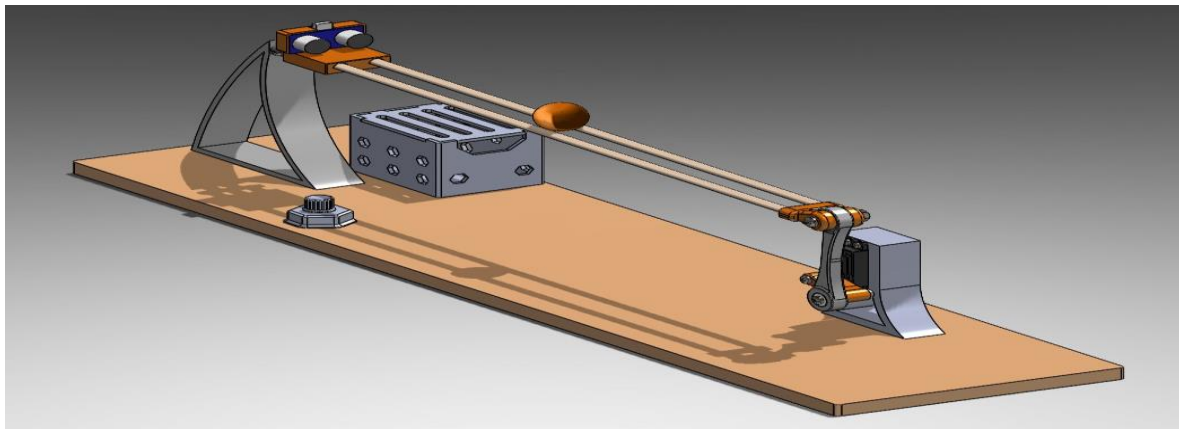


Figure 2: The CAD model by SolidWorks

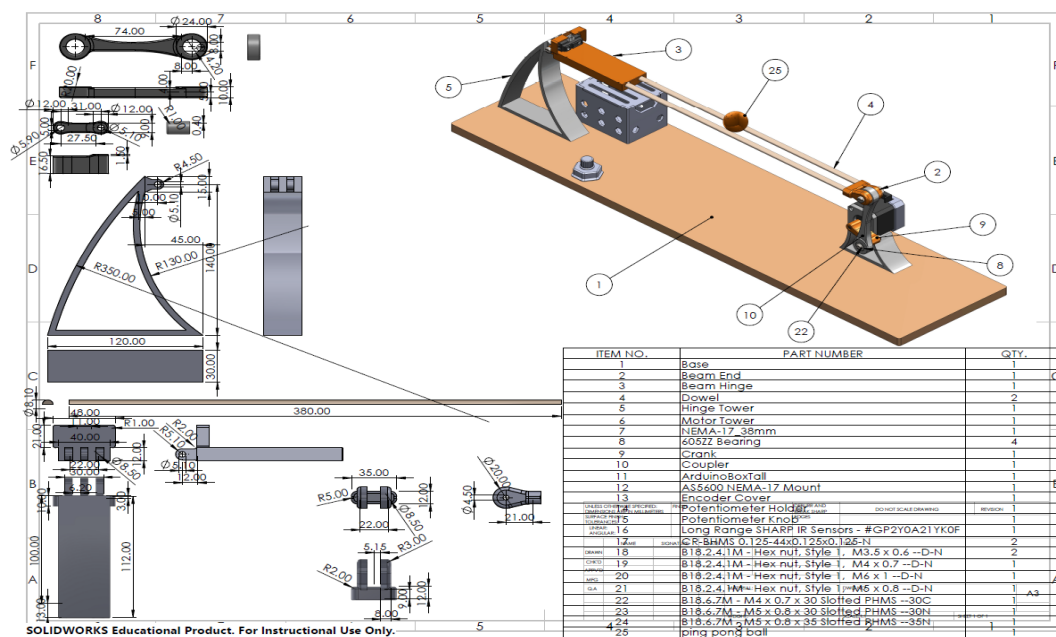


Figure 3: Shop Drawing

2.preliminary electronic/control circuit diagram

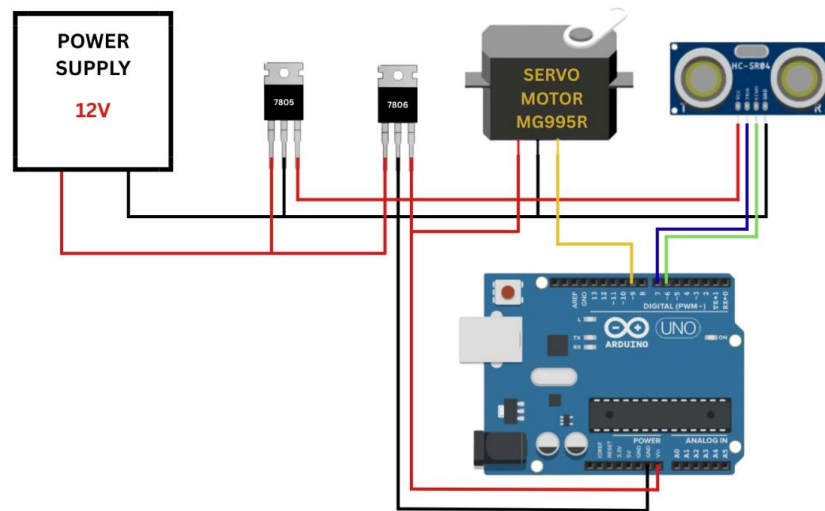


Figure 4:Circuit diagram

3.All components and their interfaces

No.	Component	Type	Function in System	Input Interface	Output Interface	Interface Type
1	Ball	Mechanical	Object to be balanced on the beam	Beam inclination angle (α)	Ball position (x)	Mechanical (Contact & Gravity)
2	Beam	Mechanical	Supports the ball and transfers tilt motion	Rotation from lever arm	Linear motion of the ball	Mechanical (Rotational $\rightarrow$ Translational)
3	Lever Arm	Mechanical	Converts servo rotation to beam tilt	Servo shaft rotation (θ)	Beam angular displacement (α)	Mechanical (Rotational Linkage)

4	Servo Motor (MG995R)	Actuator	Controls beam angle	PWM signal from Arduino	Shaft rotation (θ)	Electrical (PWM) → Mechanical
5	Ultrasonic Sensor (HC-SR04)	Sensor	Measures ball position on beam	Trigger pulse from Arduino	Echo pulse proportional to distance	Digital Electrical (Trigger/Echo)
6	Arduino Uno	Controller	Main control and signal processing unit	Sensor signals (Echo), Power (5V)	PWM to servo, Trigger to sensor	Digital Electrical
7	Control Algorithm (PID)	Software	Stabilizes ball position	Ball position error	Control signal (PWM duty cycle)	Logical / Software Interface
8	Power Supply (12V DC)	Electrical	Provides power to system	AC/DC input	12V DC output	Electrical Power
9	Voltage Regulator (7805)	Electrical	Steps down voltage for logic	12V DC	5V DC	Electrical Power
10	Voltage Regulator (7806)	Electrical	Steps down voltage for servo	12V DC	6V DC	Electrical Power

11	Mechanical Base & Frame	Structural	Supports all components	Static loads	Structural stability	Mechanical
12	Wiring & Connectors	Electrical	Signal & power transmission	Electrical signals	Electrical signals	Electrical

Mechanical Design, Kinematics, and FEA

System Overview

The finalized mechanical design of the balanced beam system consists of a servo-actuated crank-link mechanism that converts the rotational motion of the MG995 servo motor into a controlled angular displacement of the beam. The beam is hinged at one end and supports a rolling ball used to demonstrate balancing and control concepts.

2. Geometry & Dimensions

Component	Dimension
Crank length	4.2 cm
Link length	11.2 cm
Beam length	35 cm
Beam cross-section	Semi-circular, Ø 8 mm
Hinge height	14 cm
Servo rotation range	0–180°
Beam rotation range	±10° (approx.)

Note: All dimensions were selected to avoid singular configurations and mechanical interference.

Material Selection

Component	Material	Reason
Beam	PLA	Lightweight, sufficient strength, easy fabrication
Link & Crank	PLA	Low inertia, ease of manufacturing
Base	Wood	Structural stability

Note: PLA provides adequate strength with a high factor of safety while minimizing weight.

Kinematic Relation

$\theta_s \rightarrow$ Servo motor

$\phi \rightarrow$ Link Angle

$\theta_b \rightarrow$ Beam angle

Crank length (r)=.042m

Beam Length (l_b)=.35m

Link Length (L)=.112m

Links Analysis

Position Analysis

$$r \cos(\theta_s) + L \cos(\phi) = l_b \cos(\theta_b)$$

Horizontal direction

$$r \sin(\theta_s) + L \sin(\phi) = l_b \sin(\theta_b)$$

Vertical direction

$$\theta_b = \tan^{-1} \frac{r \sin(\theta_s) + L \sin(\phi)}{r \cos(\theta_s) + L \cos(\phi)}$$

Velocity Analysis

$$r \dot{\theta}_s \cos(\theta_s) + L \dot{\phi} \cos(\phi) = l_b \dot{\theta}_b \cos(\theta_b)$$

$$r \dot{\theta}_s \sin(\theta_s) + L \dot{\phi} \sin(\phi) = l_b \dot{\theta}_b \sin(\theta_b)$$

Acceleration Analysis

$$r \ddot{\theta}_s (-\sin(\theta_s)) + L \ddot{\phi} (\sin(\phi))$$

Ball analysis

Position

$$Y_{ball} = x \sin(\theta_b)$$

Velocity

$$\dot{Y}_{ball} = x \cos(\theta_b) \dot{\theta}_b$$

Accelerate

$$Y_{ball}''' = -\sin\theta_b \dot{\theta}_b^2 + \cos\theta_b \ddot{\theta}_b$$

Dynamics

$$W_{ball} = mg$$

$$W_{beam} = m_{beam}g$$

$$I_{beam} = \frac{1}{3} m_b l_b^2 = \frac{1}{3} * .011 * (.35)^2 = 4.5 \times 10^{-4}$$

$$\sum \tau = I \ddot{\theta}_b$$

$$\tau_{ball} = mgx \cos(\theta_b)$$

$$\tau_{beam} = m_{beam}g \frac{l}{2} \cos(\theta_b)$$

$$\therefore \tau_{eq} - g \cos(\theta_b) (m_{ball}x + m_{beam} \frac{l}{2}) = I_{beam} \ddot{\theta}_b$$

$$\text{at } \theta_b \text{ very small} \rightarrow \cos(\theta_b) = 1$$

$$\text{at } \theta = 0$$

$$\tau_{max} = 9.81 * (.003x + .011 * .175)$$

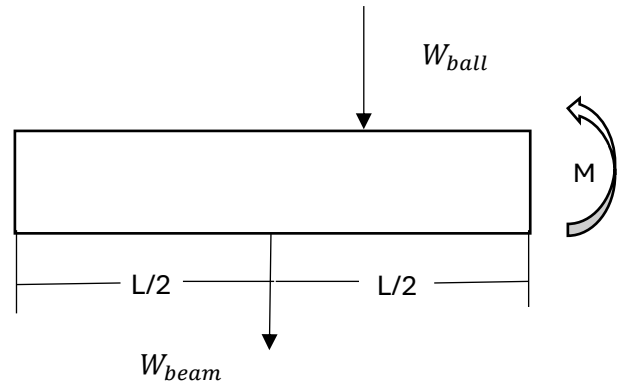
$$= .02943x + .0189333$$

$$\text{at } x = .35m$$

$$\tau = .036 \cong .04 N/m$$

We select MG995 Servo motor

Max Torque for Servo $\approx 1.1 N/m$ ($\therefore$ Suitable for system)



Stress Analysis

$$M_{max} = m_{ball}gx + m_{beam}\frac{l}{2}$$

$$I = .11r^4 = .11(.004)^4 = 2.8 \times 10^{-11}m^4$$

$$\sigma_{beam} = \frac{M_{max} * c}{I}$$

(at $x = l$)

$$M_{max} = 9.81(.003 * .35 + .11 * .175) = .029N.m$$

$$\sigma_{beam} = \frac{.029*004}{2.8*10^{-11}} = 4.1 MPa \quad (\text{bending stress})$$

$$V = (m_{ball} + m_{beam})g = (.003 + .011) * 9.81 = .137N$$

$$\tau_{max} = \frac{4V}{3A}$$

$$A = \frac{\pi r^2}{2}$$

$$\tau_{max} = 1.1 MPa \quad (\text{Shear stress})$$

$$\sigma_{eq} = \sqrt{\sigma_{beam}^2 + 3\tau_{max}^2} = 4.1 MPa$$

Summary

Stress Analysis Summary

- Maximum bending stress:

$$\sigma_{max} \approx 4.1 MPa$$

- PLA yield stress:

$$\sigma_y \approx 50 MPa$$

- Factor of Safety:

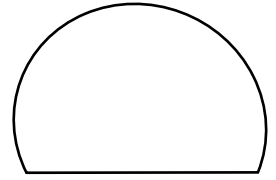
$$FOS \approx 12$$

Design is structurally safe.

Actuator Verification

- Required torque:

$$\tau_{req} \approx 0.04 N \cdot m$$



Beam Cross Section 8Ø

- MG995 available torque:

$$\tau_{stall} \approx 1.1 \text{ N} \cdot \text{m}$$

Servo motor selection is adequate with a large torque margin.

Finite Element Analysis (FEA)

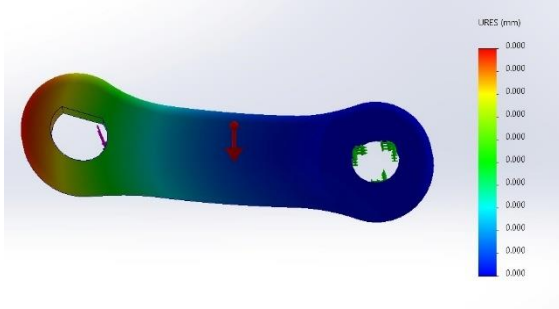


Figure 5: Total displacement (URES) of the connecting link

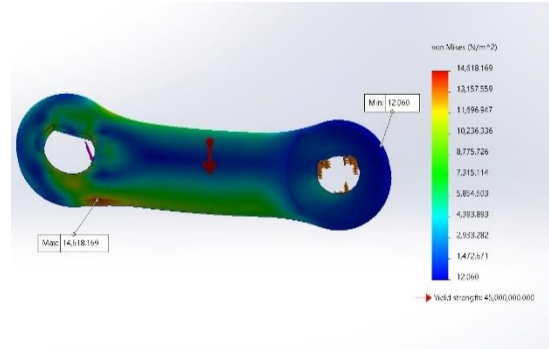


Figure 6: von Mises stress distribution of the connecting link under applied load

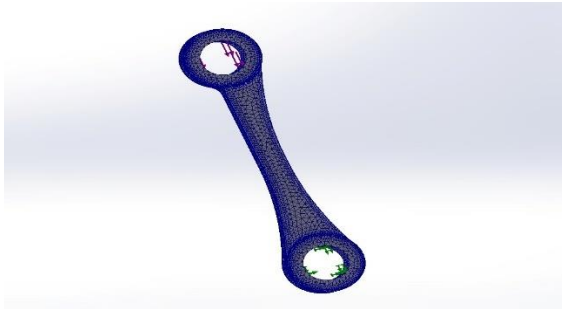


Figure 7: Finite element mesh and boundary conditions of the connecting link

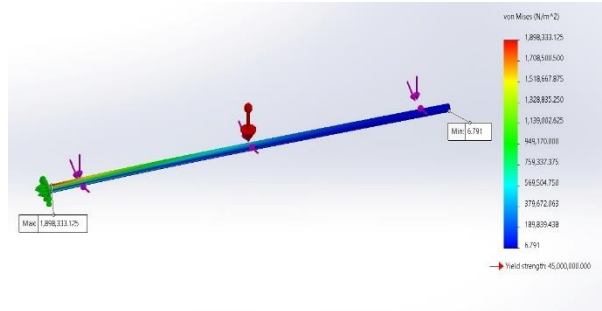


Figure 8 : von Mises stress distribution of the beam under applied loading

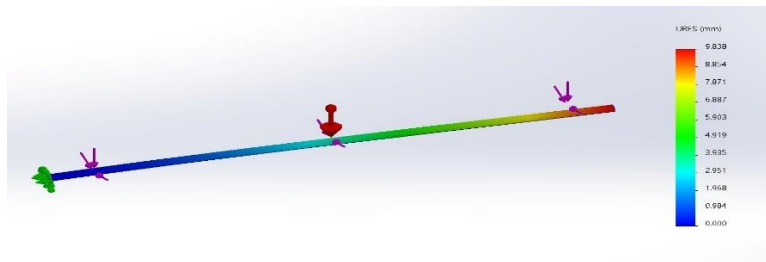


Figure 9: Total displacement (URES) distribution of the beam under applied loading

Figure 5 illustrates the total displacement (URES) distribution of the connecting link under the applied load. One end of the link is constrained, while a downward force is applied near the centre to represent the transmitted load from the crank–servo mechanism. The displacement increases gradually from the fixed end toward the free end, reaching its maximum value at the

unconstrained side. The smooth colour gradient indicates uniform deformation without localized instability. The overall displacement magnitude is very small, confirming that the link has sufficient stiffness and is suitable for use in the ball and beam system.

Figure 6 shows the **von Mises stress distribution** of the connecting link obtained from finite element analysis. One end of the link is constrained, while a downward load is applied to simulate the force transmitted from the servo-driven crank mechanism. The stress is mainly concentrated around the hole regions due to stress concentration effects caused by pin connections. The maximum von Mises stress reaches approximately **14.6 MPa**, which is significantly lower than the material yield strength of **45 MPa**. This confirms that the connecting link operates safely without risk of plastic deformation under the applied loading conditions.

Figure 7 shows the finite element model of the connecting link, including the applied mesh and boundary conditions. A triangular mesh is used to accurately represent the complex geometry of the link, with finer elements around the holes where stress concentration is expected. The lower hole is fully constrained to simulate the fixed joint, while forces are applied at the upper hole to represent the load transmitted from the servo-driven crank mechanism. This meshing and boundary condition setup ensures reliable stress and displacement results in the finite element analysis.

Figure 8 illustrates the **von Mises stress distribution** along the beam obtained from finite element analysis. The beam is constrained at one end, while external forces are applied at the mid-span and near the free end to represent the combined effect of the linkage force and the ball weight. The maximum von Mises stress, approximately **1.9 MPa**, occurs near the fixed support due to the highest bending moment. All stress values are significantly below the material yield strength of **45 MPa**, indicating that the beam operates safely under the applied loading conditions.

Figure 9 presents the **total displacement (URES) distribution** along the beam obtained from finite element analysis. The beam is fixed at one end, while downward forces are applied at the mid-span and near the free end to represent the effect of the ball weight and the linkage force. The displacement increases gradually from the fixed support toward the free end, reaching a maximum value of approximately **9.8 mm** at the free end. This deformation pattern is consistent with classical beam bending behaviour and remains within acceptable limits for the ball and beam control system.

FEA Summary

The finite element analysis confirms that:

- Maximum stress in all components is well below the yield strength.
- Displacements are within acceptable limits for accurate ball position control.
- Stress concentrations occur mainly at holes and fixed supports, as expected.
- The mechanical design is structurally safe and suitable for the ball and beam system driven by the MG955 servo motor

Final Mechanical Model Validation

The final mechanical design satisfies:

- Kinematic requirements
- Dynamic and torque constraints
- Stress and deflection limits
- Manufacturing feasibility

The system is therefore mechanically complete and ready for control implementation and experimental testing.

The mechanical design of the balanced beam system has been finalized after verifying kinematic performance, actuator capability, and structural integrity. The design meets all functional and safety requirements and provides a reliable platform for control system development.

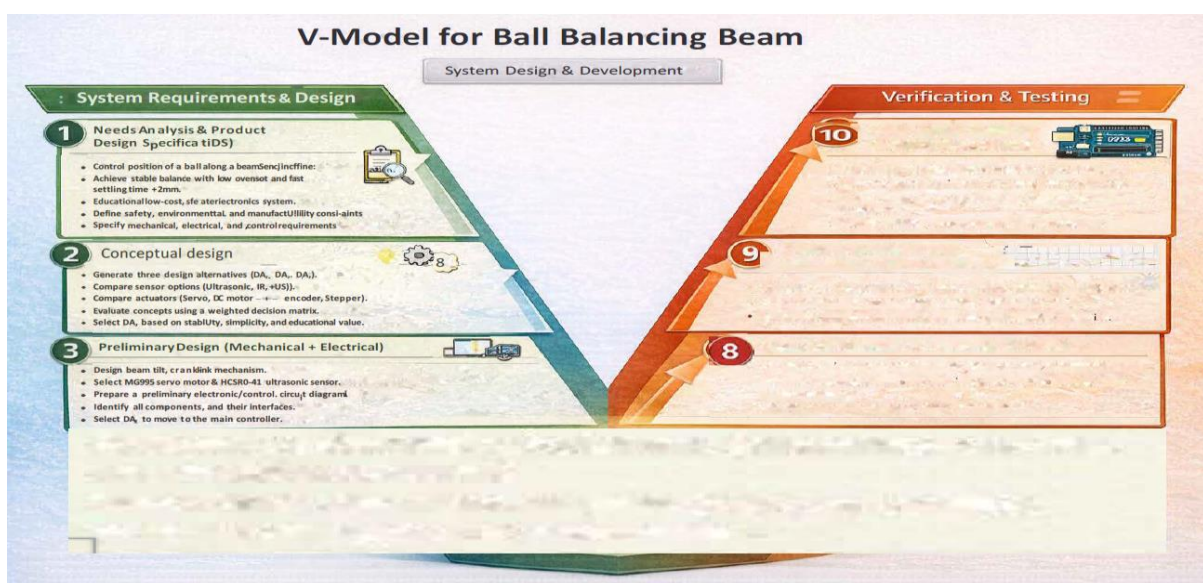


Figure 10: Left Side of the V-model

Mathematical Modeling

Motor Modeling:

$$V_a = L_a \dot{I}_a(t) + R_a I_a(t)$$

$$T_m(t) = k_t I_a(t)$$

$$T_m(t) = j_e \theta''_m + D_e \theta'_m$$

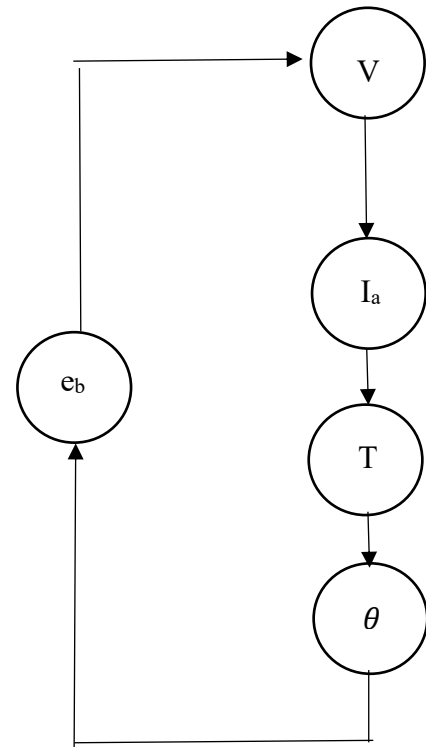
$$e_b = k_e \theta'$$

by take LaPlace and sub. all the equations:

$$\therefore \frac{I(s)}{V_a(s)} = \frac{1}{L_a s + R_a}$$

$$\therefore \frac{T_m(s)}{I_a(s)} = K_T$$

$$\therefore \frac{\theta_m(s)}{T_m(s)} = \frac{1}{s(j_e s + D_s)}$$



By putting the equations in Simulink

$$G = \frac{K_T}{S(L_a J_e S^2 + (J_e R_a + D_e L_a)S + R_a D_e)}$$

System Equations [\[10\]](#)

The second derivative of the input beam angle α affects the second derivative of the ball position r . However, this contribution is neglected for simplicity. The Lagrangian equation of motion of the ball on the beam is given by:

$$0 = \left(\frac{J}{R^2} + m \right) \ddot{r} + mg \sin \alpha - m r \dot{\alpha}^2$$

Linearizing this equation about the equilibrium beam angle $\alpha = 0$, the following linear approximation of the system is obtained:

$$\left(\frac{J}{R^2} + m \right) \ddot{r} = -mg\alpha$$

The relationship between the beam angle α and the gear (servo) angle θ can be approximated by the linear relation:

$$\alpha = \frac{d}{L} \theta$$

By Substituting Equation:

$$\left(\frac{J}{R^2} + m\right) \ddot{\theta} = -mg \frac{d}{L} \theta$$

Transfer Function

Taking the Laplace transform of Equation (4), assuming zero initial conditions, gives:

$$\left(\frac{J}{R^2} + m\right) s^2 R(s) = -mg \frac{d}{L} \Theta(s)$$

Rearranging, the transfer function from the gear angle $\Theta(s)$ to the ball position $R(s)$ is obtained as:

$$P(s) = \frac{R(s)}{\Theta(s)} = \frac{mgd}{L \left(\frac{J}{R^2} + m\right) s^2} \text{ m/rad}$$

For Our system

$m = 0.0027$ Ball mass (kg)

$L = 0.35$ Beam length (m)

$R = L - 0.02$ Distance from pivot (m)

$d = 0.042$ Lever arm length (m)

$J = 8 \times 10^{-7}$ Ball moment of inertia (kg.m²)

$g = 9.81$ Gravity (m/s²)

MATLAB Simulation

```
% =====  
% Benha University  
% Faculty of Engineering  
% Mechatronics and Automation Engineering Program  
% MAM 301 - Design of Mechatronics  
% Design Project: Ball and Beam System  
  
% Students:  
% Mahmoud Mohamed Shamekh  
% Omar Mahmoud Metwally  
% Youssef Mostafa Ayad  
% Ebrahim Magdy Ebrahim  
  
% Supervisors:  
% Dr. Amro Shafik  
% Eng. Mohamed Ashraf  
% =====  
  
clc; clear; close all;  
  
%% =====  
% Physical Parameters  
%% =====  
M = 0.0027; % Ball mass (kg)  
L = 0.35; % Beam length (m)  
R = L - 0.02; % Distance from pivot (m)  
D = 0.042; % Ball diameter (m)  
r = D/2; % Ball radius (m)  
J = 8e-7; % Ball moment of inertia (kg.m^2)  
g = -9.81; % Gravity acceleration (m/s^2)  
  
%% =====  
% Ball & Beam Model Constants  
%% =====  
A = (J/(r^2)) + M; % Effective mass term  
K = (M * g * R) / L; % System constant  
  
%% =====  
% Transfer Function  
%% =====  
s = tf('s');  
  
% Ball position / Beam angle transfer function  
P = -(K/A) * 1/(s^2);  
  
disp('Ball and Beam Plant Transfer Function:')  
P
```

```

%% =====
% Open-Loop Response (Unstable)
%% =====
figure
step(P,5)
grid on
title('Open-Loop Step Response (Unstable Ball & Beam System)')
xlabel('Time (s)')
ylabel('Ball Position (m)')

%% =====
% PID Controller Parameters
%% =====
Kp = 40;          % Proportional gain
Ki = 5;           % Integral gain
Kd = 12;          % Derivative gain

% PID controller (transfer function form, without scalar gain)
C0 = Kp + Ki/s + Kd*s;

%% =====
% Root Locus with Variable Gain K
%% =====
figure
rlocus(C0 * P)
grid on
title('Root Locus of Ball and Beam System with PID Controller')
sggrid(0.7, []) % Desired damping ratio

%% =====
% Gain Selection
%% =====
K_gain = 1;       % Selected from root locus (adjust if needed)

C_final = K_gain * C0;
T_final = feedback(C_final * P, 1);

%% =====
% Closed-Loop Response
%% =====
figure
step(T_final,5)
grid on
title(['Closed-Loop Step Response with PID (K = ', num2str(K_gain), ')'])
xlabel('Time (s)')
ylabel('Ball Position (m)')

```

```

%% =====
% Effect of PID Parameters
%% =====
% ---- Effect of Kp ----
Kp_vals = [20 40 70];
figure; hold on
for kp = Kp_vals
    Ctemp = kp + Ki/s + Kd*s;
    Ttemp = feedback(Ctemp * P, 1);
    step(Ttemp,5)
end
legend('Kp = 20', 'Kp = 40', 'Kp = 70')
grid on
title('Effect of Proportional Gain Kp')
xlabel('Time (s)')
ylabel('Ball Position (m)')

% ---- Effect of Ki ----
Ki_vals = [0 5 15];
figure; hold on
for ki = Ki_vals
    Ctemp = Kp + ki/s + Kd*s;
    Ttemp = feedback(Ctemp * P, 1);
    step(Ttemp,5)
end
legend('Ki = 0', 'Ki = 5', 'Ki = 15')
grid on
title('Effect of Integral Gain Ki')
xlabel('Time (s)')
ylabel('Ball Position (m)')

% ---- Effect of Kd ----
Kd_vals = [0 10 25];
figure; hold on
for kd = Kd_vals
    Ctemp = Kp + Ki/s + kd*s;
    Ttemp = feedback(Ctemp * P, 1);
    step(Ttemp,5)
end
legend('Kd = 0', 'Kd = 10', 'Kd = 25')
grid on
title('Effect of Derivative Gain Kd')
xlabel('Time (s)')
ylabel('Ball Position (m)')
%% =====
% Stability Information (Poles & Damping Table)
%% =====
[wn, zeta, poles] = damp(T_final);

ResultsTable = table( ...
    poles, ...
    zeta, ...
    wn, ...
    'VariableNames', {'Pole', 'DampingRatio', 'NaturalFrequency'} ...
);

disp('Closed-Loop Poles and Damping Information:')
disp(ResultsTable)

```


The output of the code

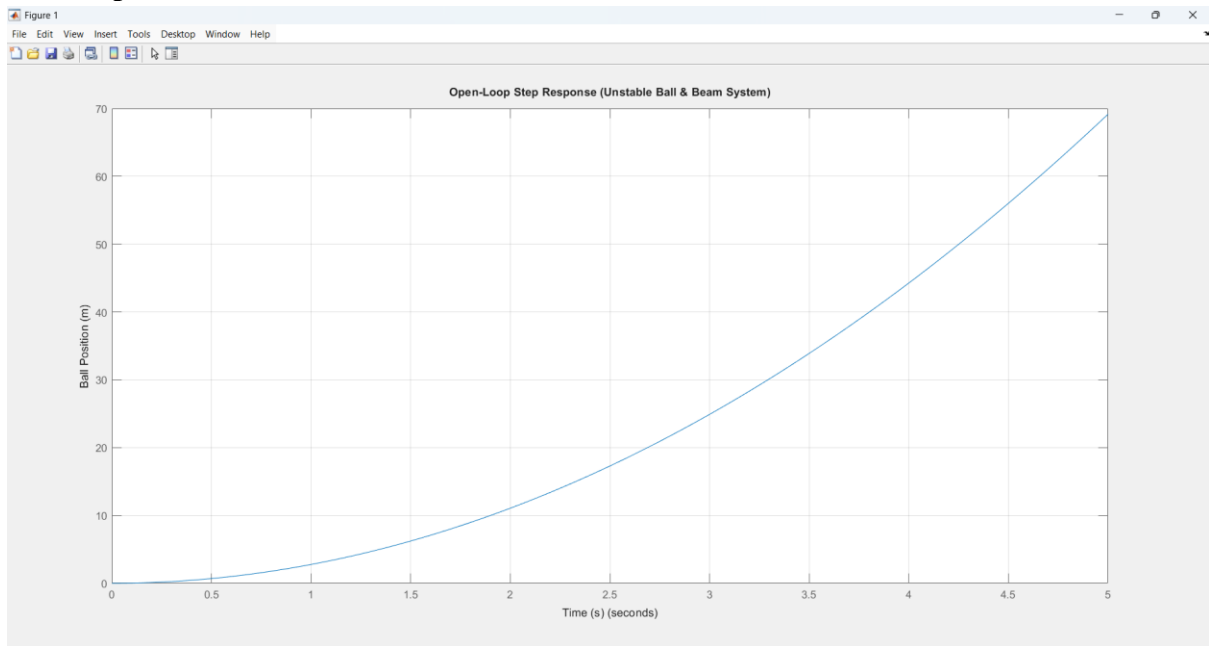


Figure 11:Open Loop Response

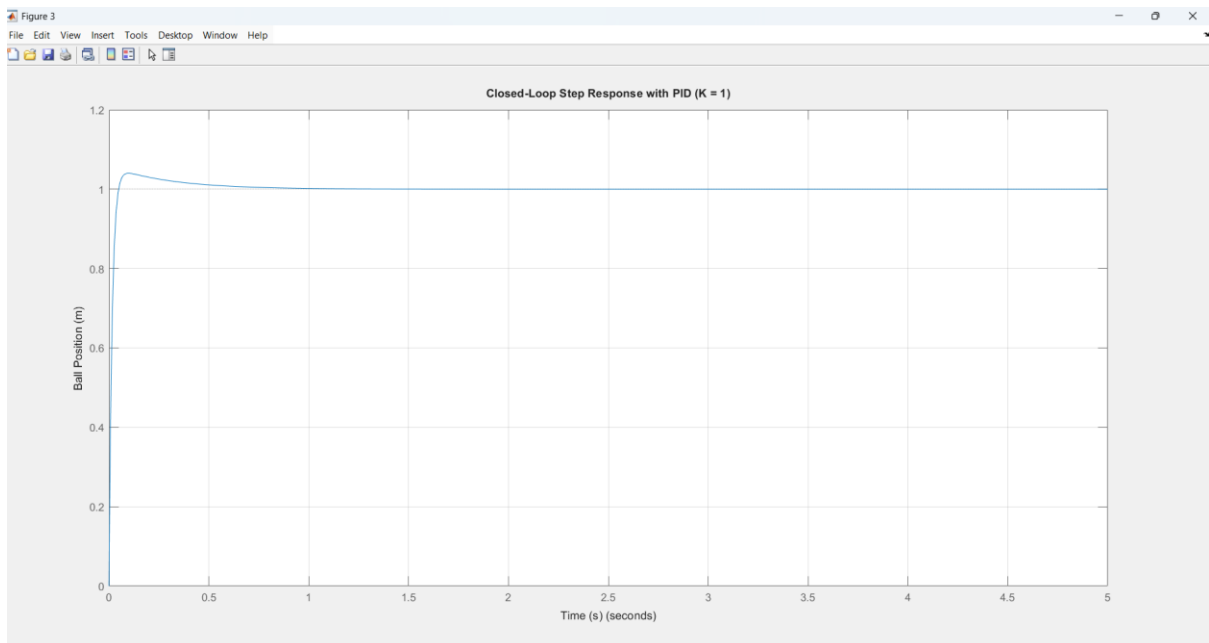


Figure 12: Closed loop Response

Ball and Beam Plant Transfer Function:

$$P(s) = \frac{5.532}{s^2}$$

Closed-Loop Poles and Damping Information:

Pole	Damping Ratio	Natural Frequency
-0.13007	1	0.13007
-3.3825	1	3.3825
-62.876	1	62.876

[Figure 11](#) illustrates the open-loop step response of the ball and beam system. The response shows that the ball position increases rapidly and continuously with time without reaching a steady value. This unbounded growth is caused by the double integrator nature of the plant, which makes the system unstable in open-loop operation. Any small input disturbance results in continuous acceleration of the ball along the beam, confirming that feedback control is essential to stabilize the system and prevent the ball from diverging.

[Figure 12](#) presents the closed-loop step response of the ball and beam system when a PID controller is applied. The response demonstrates a stable behavior where the ball position quickly reaches the desired reference value with a small overshoot and then settles smoothly without sustained oscillations. The steady-state error is effectively eliminated due to the integral action, while the derivative term provides sufficient damping. This result confirms that the selected PID controller parameters successfully stabilize the ball and beam system and significantly improve its transient and steady-state performance compared to the open-loop case.

Identify stability issues and behavior under different inputs

At $K_p = 40, K_i = 5, K_d = 10$

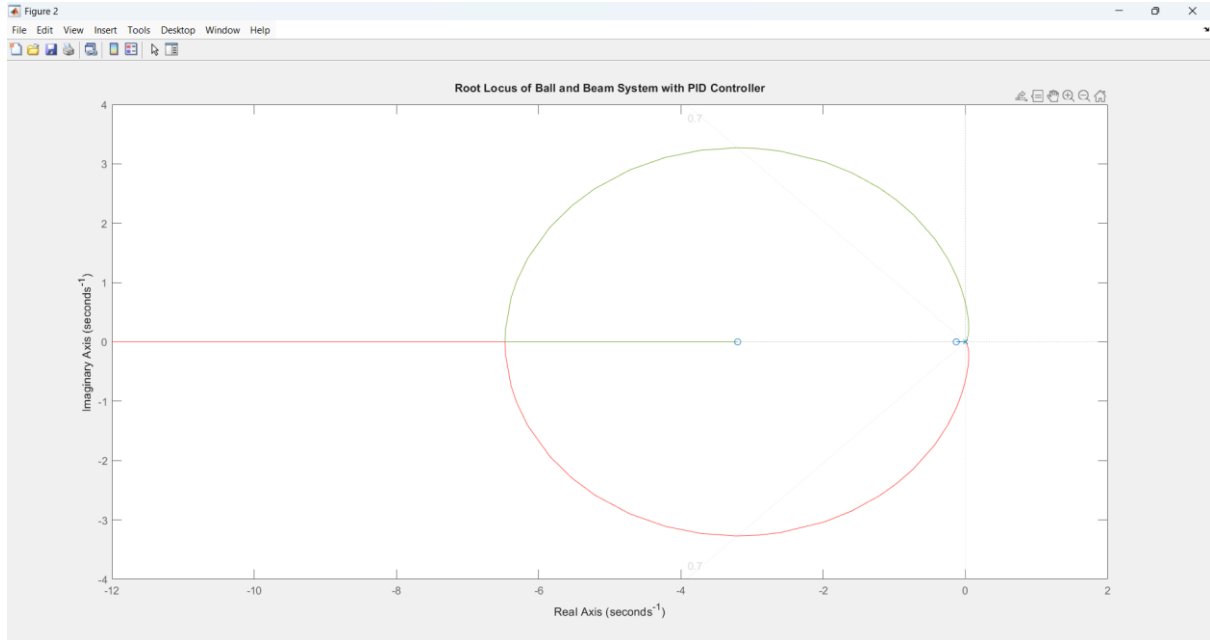


Figure 13:root locus

By applying different values of K_p, K_i, K_d to illustrate each effect

For K_p

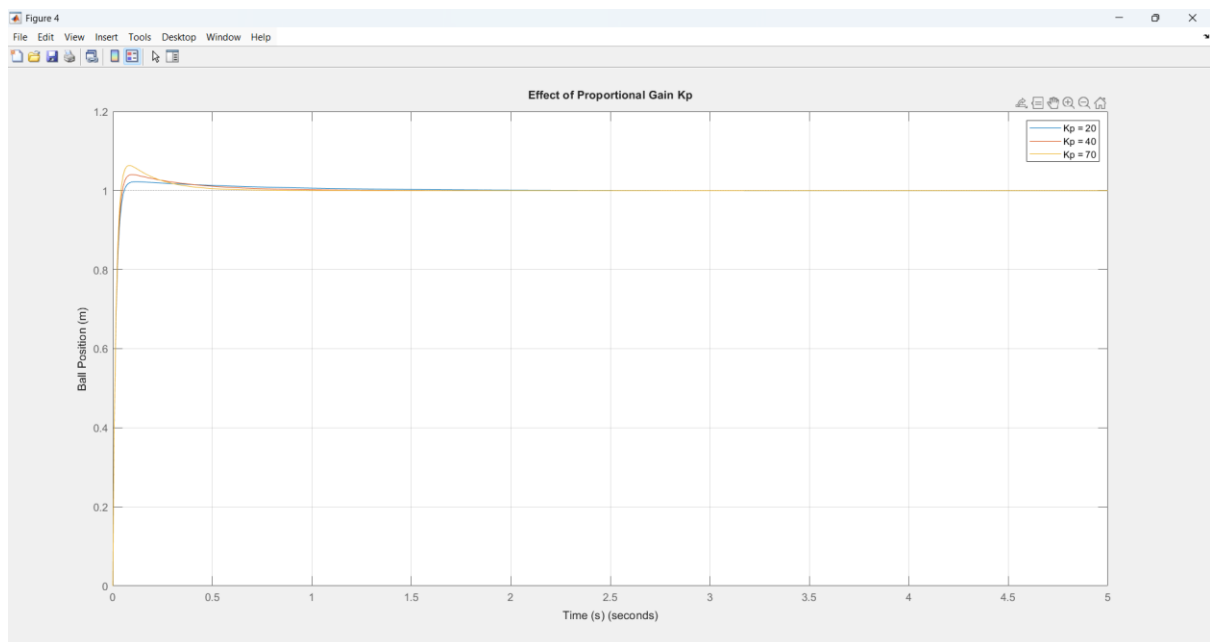


Figure 14:The effect of K_p at (20,70)

For K_i

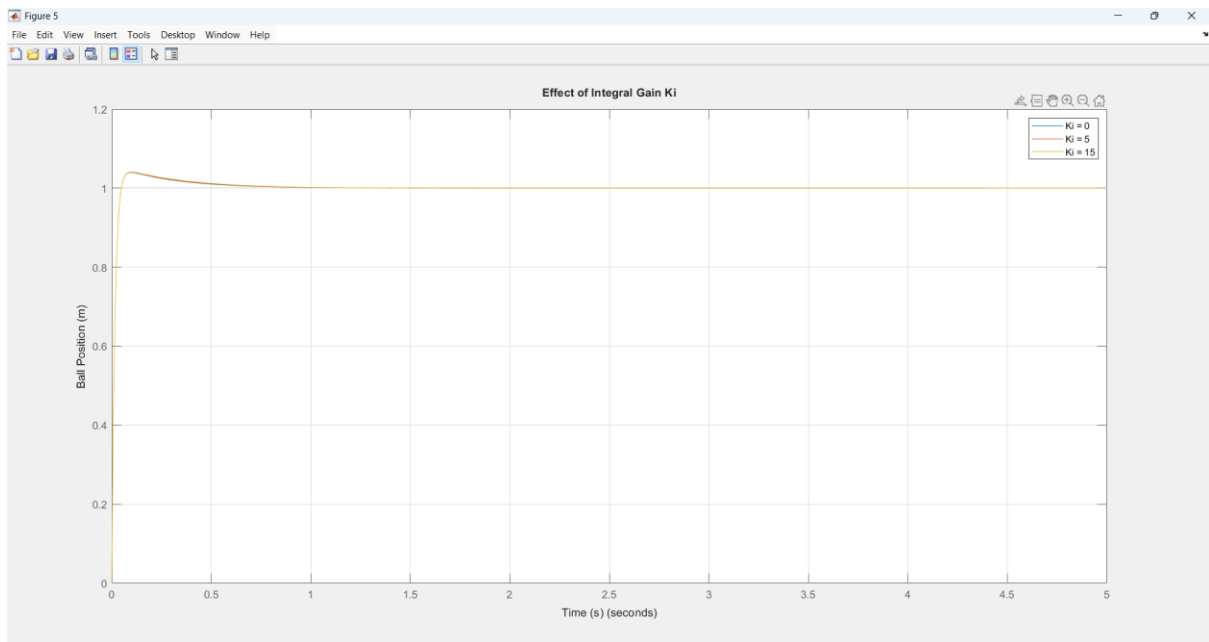


Figure 15: The effect of K_i at (0,15)

For K_d

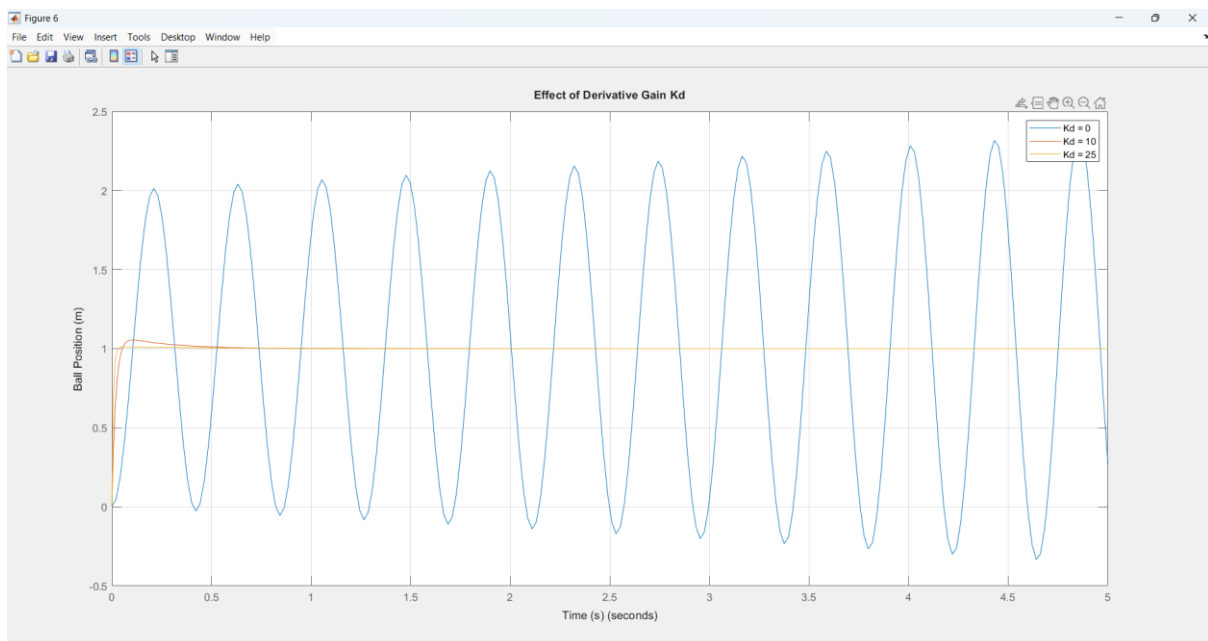


Figure 16: The effect of K_d at (0,25)

Simulink Simulation

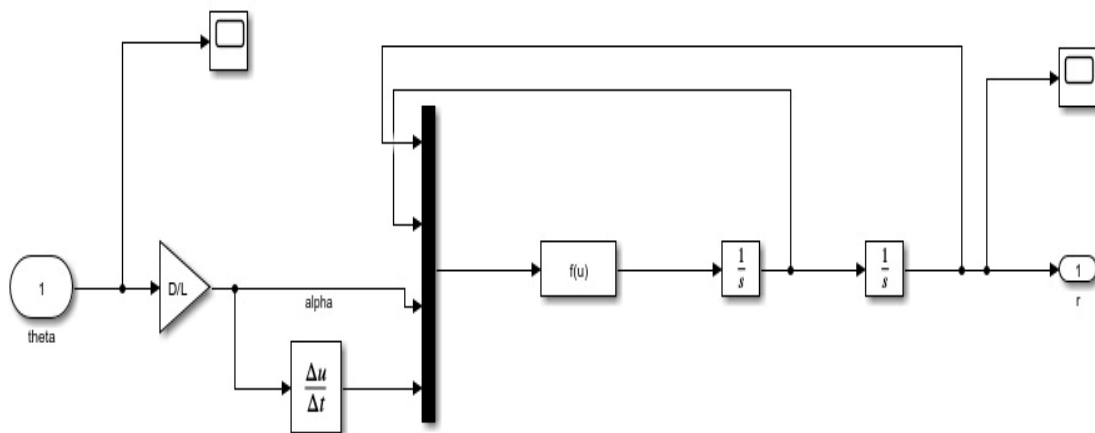


Figure 18:open loop by Simulink

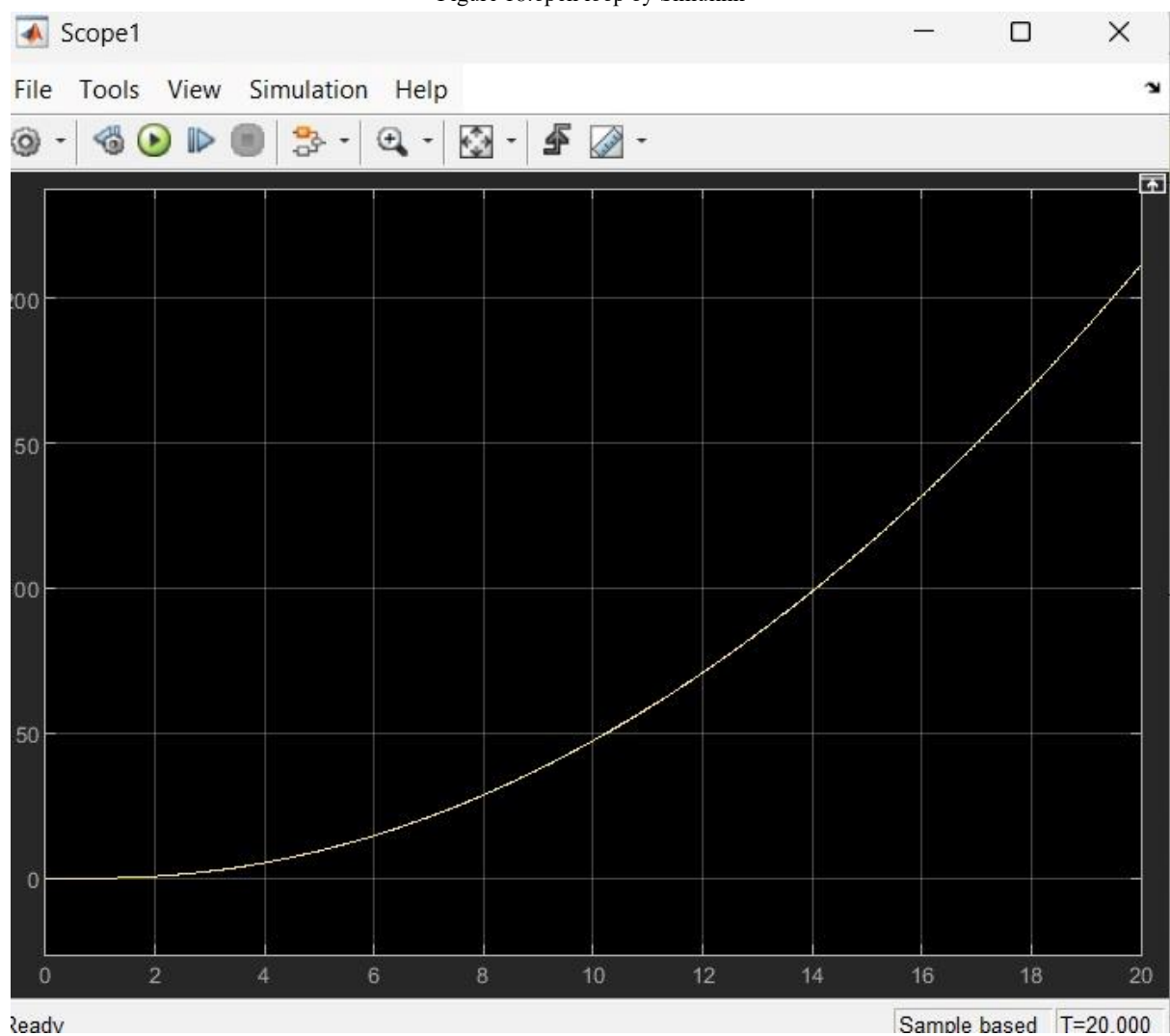


Figure 17:open loop Response

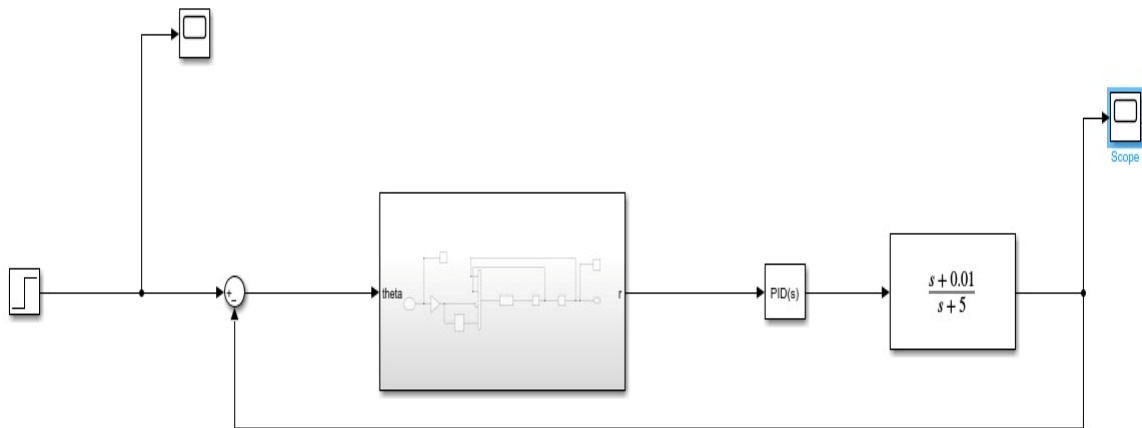


Figure 19:closed loop by Simulink



Figure 20: closed loop response

Stability Analysis and System Behavior Under Different Inputs

The open-loop step response of the ball and beam system clearly demonstrates that the system is unstable. As shown in the open-loop response figure, the ball position increases rapidly and diverges with time due to the double integrator nature of the plant. This confirms that without feedback control, the ball cannot be stabilized on the beam, and any small input leads to unbounded motion.

When the PID controller with gains $K_p = 40$, $K_i = 5$, and $K_d = 10$ is applied, the closed-loop step response shows stable and well-damped behavior. The ball position reaches the desired reference with a small overshoot and settles quickly without sustained oscillations. This indicates that the selected PID parameters successfully stabilize the system and provide acceptable transient performance.

The root locus plot further supports this observation, as all closed-loop poles are in the left half of the complex plane for the chosen gain, with a damping ratio close to the desired value ($\zeta \approx 0.7$). This confirms that the closed-loop system is stable and adequately damp.

The effect of varying the proportional gain K_p shows that increasing K_p improves the response speed but also increases overshoot. Low values of K_p result in slower responses, while excessively high values lead to more aggressive behavior. An intermediate value of $K_p = 40$ provides a good balance between fast response and acceptable overshoot.

The effect of the integral gain K_i demonstrates its role in eliminating steady-state error. When $K_i = 0$, a small steady-state error is observed. Increasing K_i reduces this error and ensures accurate tracking of the reference position. However, very large values of K_i may slightly increase overshoot and slow down the settling process.

The effect of the derivative gain K_d highlights its importance in damping the system. With $K_d = 0$, the response exhibits large oscillations and poor stability. As K_d increases, oscillations are significantly reduced, and the response becomes smoother. Higher derivative gains improve damping but must be carefully selected to avoid excessive sensitivity to noise.

Finally, the Simulink simulation results confirm the MATLAB analytical results. The open-loop Simulink model shows unstable behavior, while the closed-loop PID-controlled model stabilizes the ball position effectively. The consistency between MATLAB simulations and Simulink models validates the correctness of the mathematical model and the chosen PID control parameters.

Hardware–Software Integration and Data Visualization

In this work, the PID controller was implemented on an Arduino microcontroller using an embedded program. The Arduino was interfaced with MATLAB through serial communication, enabling real-time data exchange between the hardware controller and the simulation environment. The control signal and system output were transmitted via the serial port, where MATLAB received and processed the incoming data. A dedicated MATLAB script was developed to read the serial data stream and generate time-domain plots of the system output. This approach allowed real-time visualization and analysis of the system response, facilitating direct observation of the closed-loop behaviour and performance of the Ball and Beam system.

Arduino code

```
#include <Servo.h>
#define Umax 66
#define Umin -66
#define Umax_rad 1.151
#define Umin_rad -1.151
#define T 0.09
const int echoPin1 = 3;
const int trigPin1 = 2;
Servo servo;
// ===== Variables =====
double setpoint = 0.20;           // ثابتة (بالمتر) 20 cm
double setpoint_prec = 0.20;
double y, y_prec;
double error;
double P, I, D, U;
double I_prec = 0, D_prec = 0;
boolean Saturation = false;
```



```

// ===== PID Gains =====
double Kp = 25;
double Ki = 17;
double Kd = 140;

// ===== Functions =====
float measure_1(void);
void move_servo(int);

//=====
unsigned long t_prev = 0;
const unsigned long Ts = 10; // 10 ms = 0.01 s

//=====
// ===== Setup =====
void setup() {

    Serial.begin(9600);

    pinMode(trigPin1, OUTPUT);
    pinMode(echoPin1, INPUT);

    servo.attach(9);

    delay(1000);
    move_servo(90);
    delay(2000);

    y_prec = measure_1();
    delay(1000);
}

// ===== Loop =====
void loop() {

    // Measurement
    y = measure_1();
    y = 0.53 * y + 0.47 * y_prec;
    // Quantization 0.01 كل
    y = round(y * 100.0) / 100.0;

    // Print
    Serial.println(round(y * 100.0) / 100.0);
    // === PID (لو حابب) ===
    error = y - setpoint;
    // Error
    error = round(100 * (y - setpoint)) * 0.01;

```

```

// PID
P = Kp * error;

if (!Saturation)
    I = I_prec + T * Ki * error;

D = (Kd / T) * (y - y_prec);
D = 0.56 * D + 0.44 * D_prec;

U = P + I + round(100 * D) * 0.01;    // radians

// Saturation
if (U < Umin_rad) {
    U = Umin_rad;
    Saturation = true;
}
else if (U > Umax_rad) {
    U = Umax_rad;
    Saturation = true;
}
else {
    Saturation = false;
}

// Convert to servo angle
U = round(U * 180 / M_PI);
U = map(U, Umin, Umax, 24, 156);

// Move servo only if needed
if (U < 83 || U > 95 || abs(error) > 0.02)
    move_servo(round(U));

delay(24);

// Update previous values
I_prec = I;
y_prec = y;
D_prec = D;
}

// ===== Ultrasonic Sensor =====
float measure_1(void) {

    long elapsed_time = 0;
    float distance = 0;

    digitalWrite(trigPin1, LOW);
    delayMicroseconds(10);

```

```

digitalWrite(trigPin1, HIGH);
delayMicroseconds(10);

digitalWrite(trigPin1, LOW);

elapsed_time = pulseIn(echoPin1, HIGH);
distance = (float)elapsed_time / 58.2;

delay(30);

if (distance > 42) distance = 43;
else if (distance < 0) distance = 0;

return 0.01 * (distance - 1.5 + 0.5); // meters
}

// ===== Servo =====
void move_servo(int u) {
    u = constrain(u, 0, 180);
    servo.write(u);
}

```

MATLAB Code

```

clc;
clear;
close all;

% serial
s = serialport("COM11", 9600);
configureTerminator(s, "LF");
flush(s);

% drawing
figure;
h = animatedline('LineWidth', 2);
grid on;
xlabel('Sample');
ylabel('Distance (m)');
title('Ultrasonic Distance (0.01 m resolution)');

k = 0;

% storage data
xData = [];
yData = [];
constData = []; % constant distance 0.2m

C = 0.20;

```

```

% readings
while ishandle(h)
    if s.NumBytesAvailable > 0
        data = readline(s);
        y = str2double(data);

        if ~isnan(y)
            k = k + 1;

            xData(k,1) = k;
            yData(k,1) = y;
            constData(k,1) = C;

            % plot
            addpoints(h, k, y);
            drawnow limitrate;
        end
    end
end
end

```

The output of the two codes give us plot for Output With time And The Desired distance

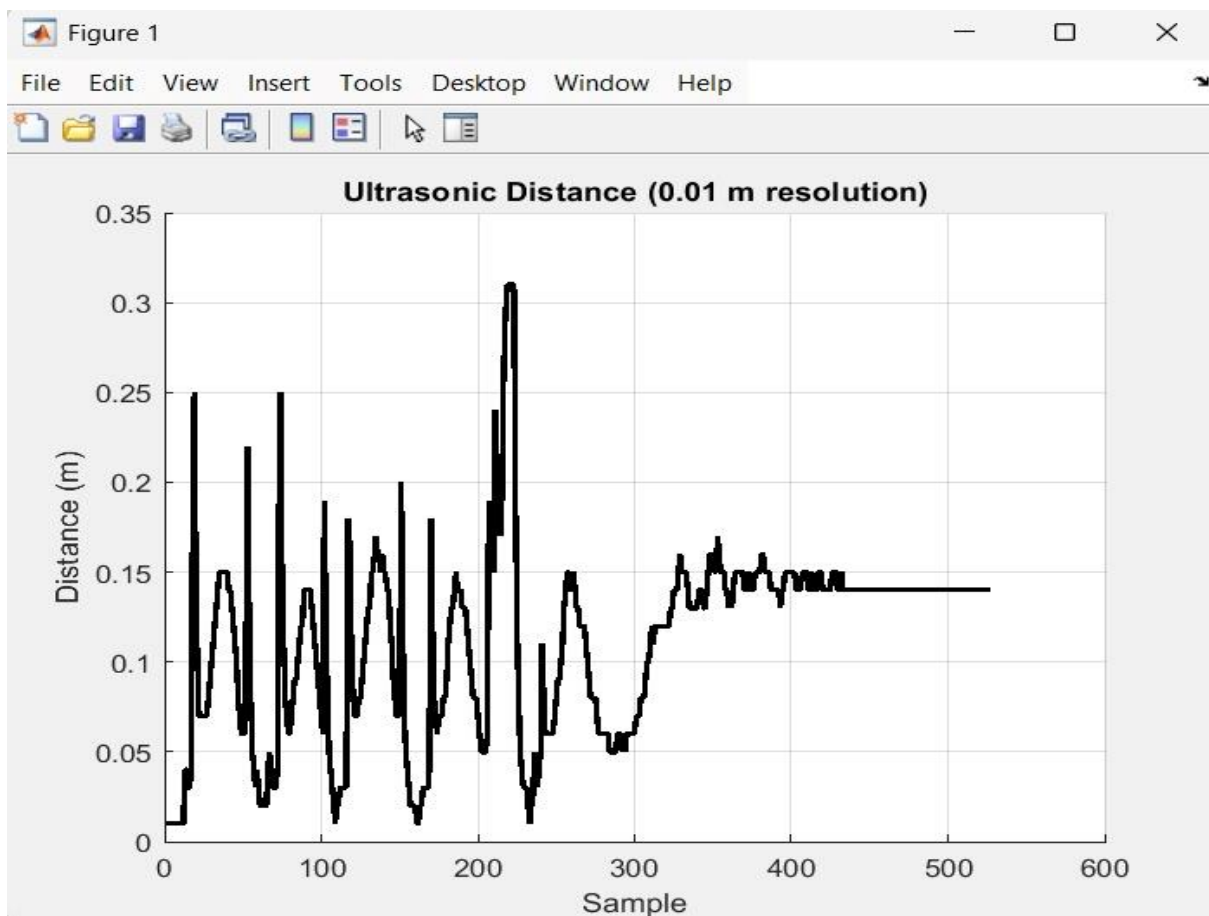


Figure 21:Output Distance with time

System Identification & Parameter Estimation

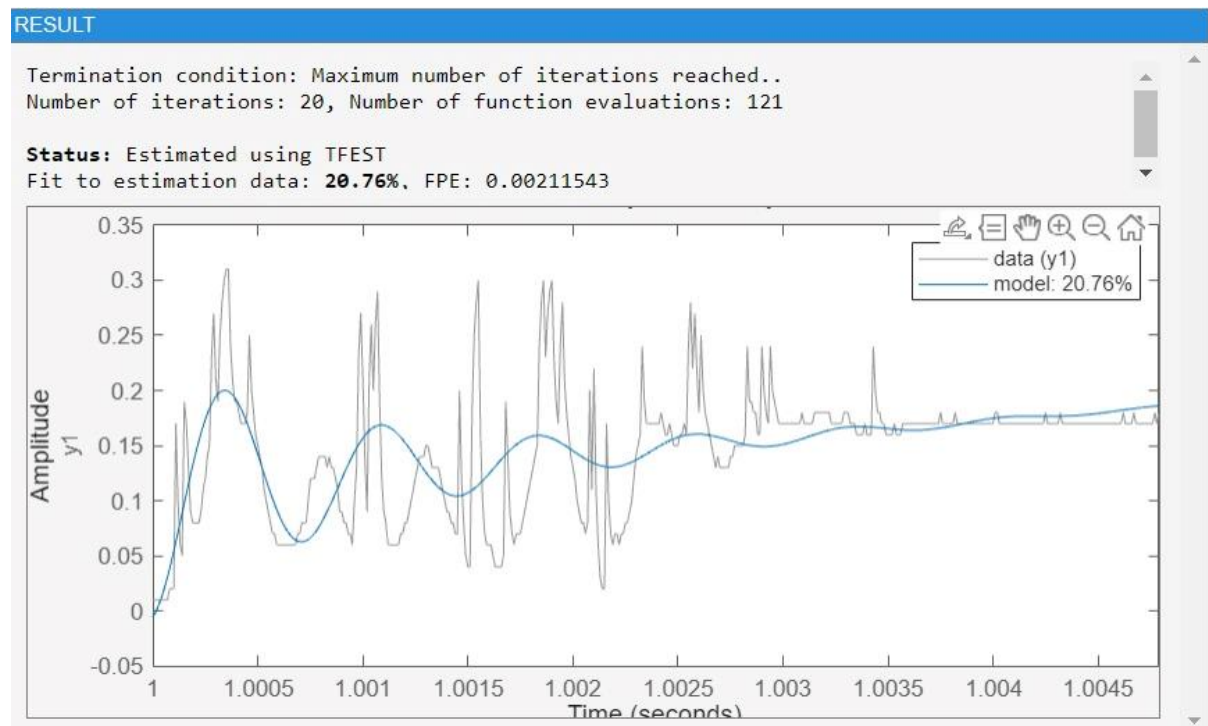


Figure 22: output distance -Input time

In this case

$$P(s) = \frac{1.625s + 2.959}{s^2 + 15.09s + 12850}$$

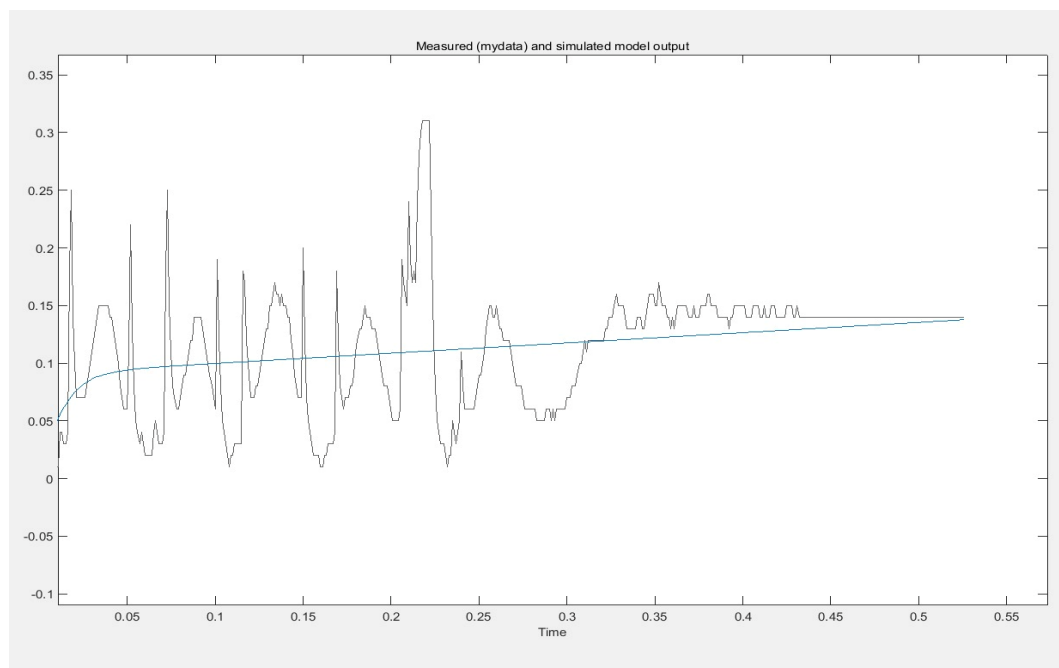


Figure 23:output & input distance

In this case

$$P(s) = \frac{25.05s + 39.11}{s^2 + 87.88s + 1.494 \times 10^{-10}}$$

We use system identification Toolbox To show these data

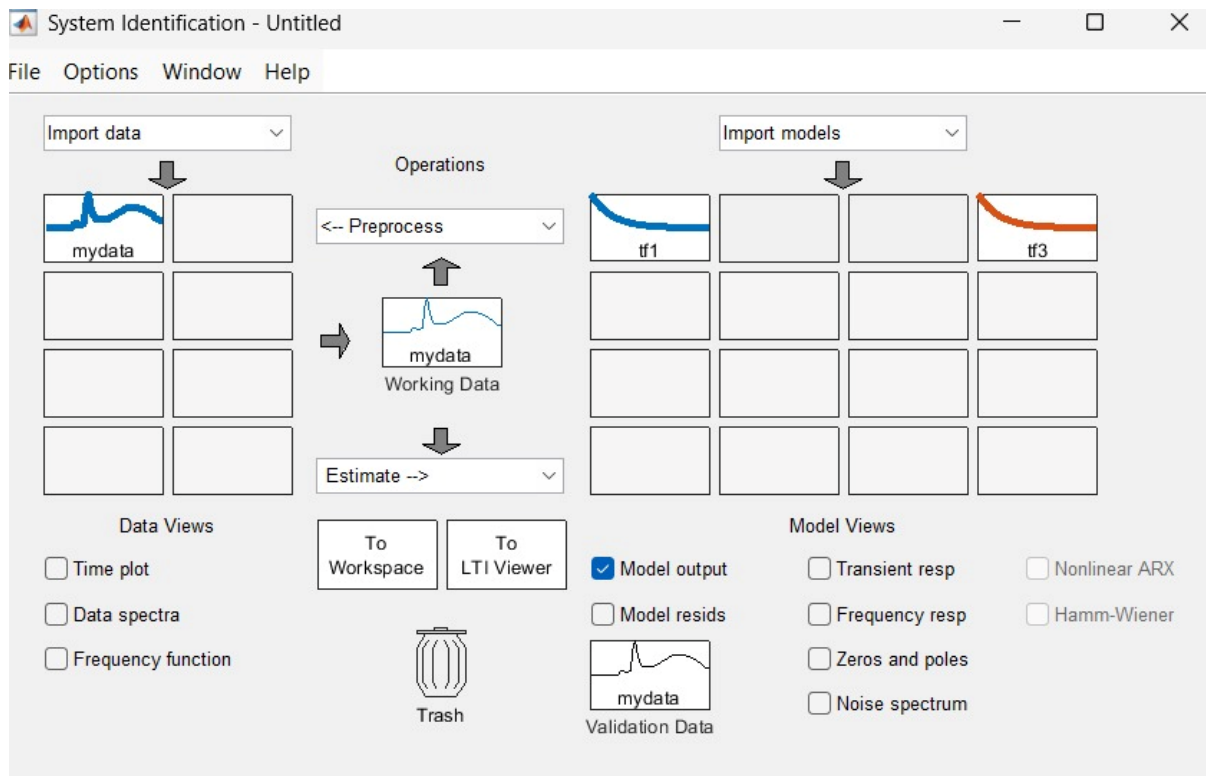


Figure 25:The toolbox

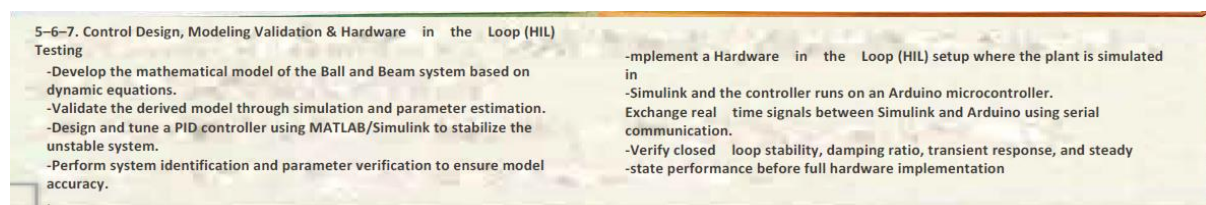


Figure 24:bottom of the V-model

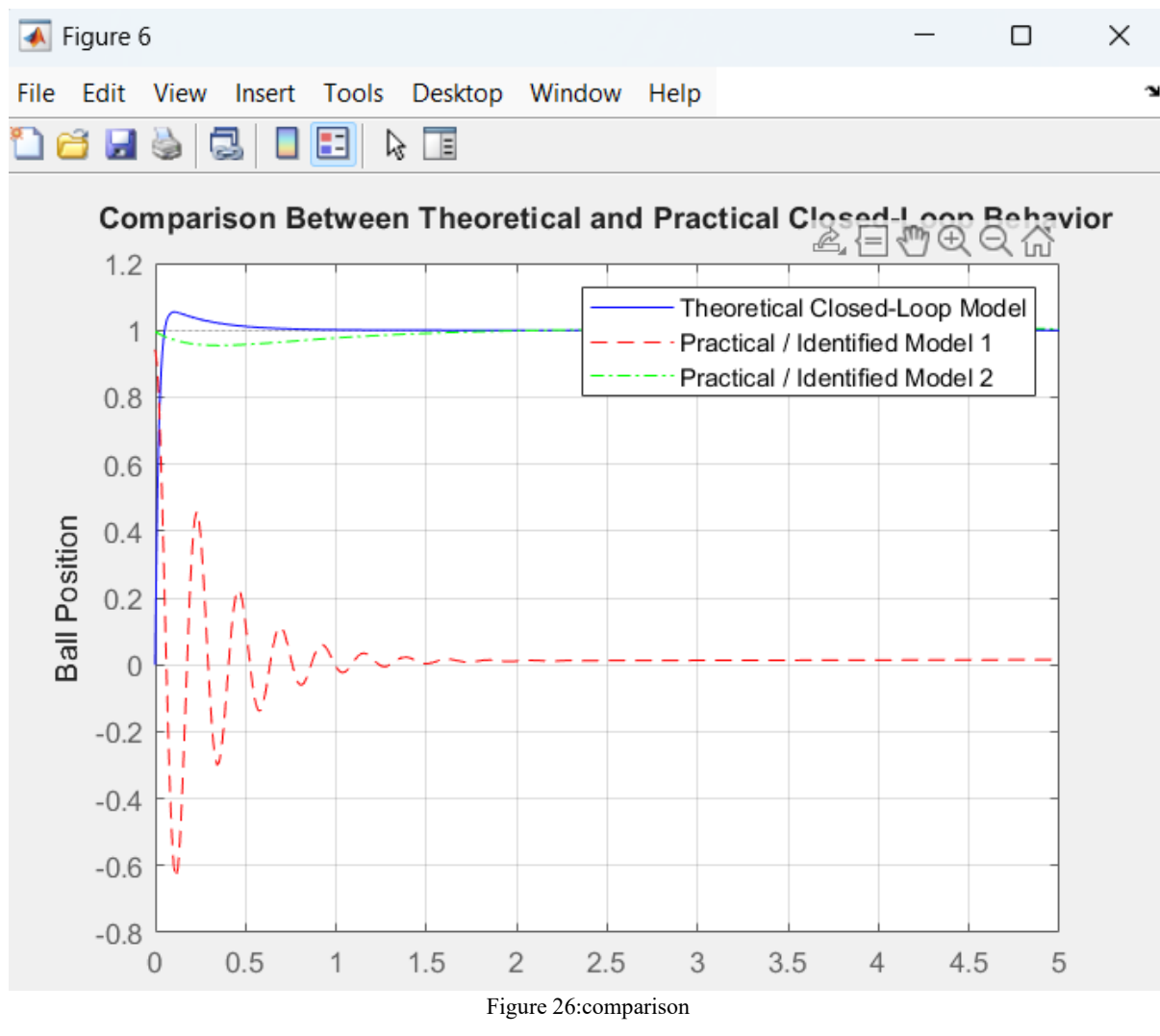
Comparison Between Simulation and Practical Work

[Figure 23](#), [figure 22](#) presents a comparison between the theoretical closed-loop response obtained from the mathematical model and two practical closed-loop responses derived from experimentally identified system models. The theoretical model exhibits a fast and well-damped response with a small overshoot and a short settling time, indicating that the

designed PID controller achieves the desired stability and transient performance under ideal modeling assumptions.

In contrast, the practical identified models show noticeable deviations from the theoretical response. Practical Model 1 exhibits oscillatory behavior with a larger overshoot and longer settling time, which can be attributed to unmodeled dynamics, actuator nonlinearities, and delays in the real system. Practical Model 2 shows a slower response with reduced oscillations but a slight steady-state deviation, reflecting the influence of parameter variations, friction, and sensor inaccuracies.

The observed differences between the simulation and practical responses are mainly caused by modeling errors, sensor noise, friction effects, and time delays that are not fully captured in the theoretical model. Despite these differences, the overall trends of the responses confirm that the PID controller successfully stabilizes the Ball and Beam system in both simulated and practical scenarios. This comparison validates the effectiveness of the control design while



highlighting the importance of experimental identification and real-world testing in mechatronic system development

Comparison between theoretical and practical behavior

```
%% ===== Performance Analysis =====

% Step information
info_theoretical = stepinfo(T);
info_practical1 = stepinfo(T_practical);
info_practical2 = stepinfo(T_practical2);

% Steady-state values
ss_theoretical = dcgain(T);
ss_practical1 = dcgain(T_practical);
ss_practical2 = dcgain(T_practical2);

% Steady-state error (for unit step input)
ess_theoretical = abs(1 - ss_theoretical);
ess_practical1 = abs(1 - ss_practical1);
ess_practical2 = abs(1 - ss_practical2);

% Display results
fprintf('\n==== Closed-Loop Performance Comparison ==== \n');

fprintf('\nTheoretical Model: \n');
fprintf('Overshoot = %.2f %% \n', info_theoretical.Overshoot);
fprintf('Settling Time = %.3f s \n', info_theoretical.SettlingTime);
fprintf('Steady-State Error = %.5f \n', ess_theoretical);

fprintf('\nPractical Model 1: \n');
fprintf('Overshoot = %.2f %% \n', info_practical1.Overshoot);
fprintf('Settling Time = %.3f s \n', info_practical1.SettlingTime);
fprintf('Steady-State Error = %.5f \n', ess_practical1);

fprintf('\nPractical Model 2: \n');
fprintf('Overshoot = %.2f %% \n', info_practical2.Overshoot);
fprintf('Settling Time = %.3f s \n', info_practical2.SettlingTime);
fprintf('Steady-State Error = %.5f \n', ess_practical2);
```

Model	Overshoot (%)	Settling Time (s)	Steady-State Error
Theoretical Model	5.43	0.364	0.00000
Practical Model 1	0.00	1005.137	0.00000

Parameter	Value			Units
(R)	2.5			Ω
(K_t)	5.8857×10^{-3}			Nm/A
(K_ω)	169.9029			rad/V
(J_m)	6.3173×10^{-7}			kg·m ²
(N)	275.6923			—
(η)	0.81156			—
(b_m)	2.49797×10^{-7}			N·m·s
(K_P)	16.6742			—
Practical Model 2	0.77	0.000	0.00000	

System Parameters

Transfer Function Comparison

Theoretical	System identification	
$P(s) = \frac{5.532}{s^2}$	Distance to Distance	Time to distance
	$P(s) = \frac{25.05s + 39.11}{s^2 + 87.88s + 1.494 \times 10^{-10}}$	$P(s) = \frac{1.625s + 2.959}{s^2 + 15.09s + 12850}$

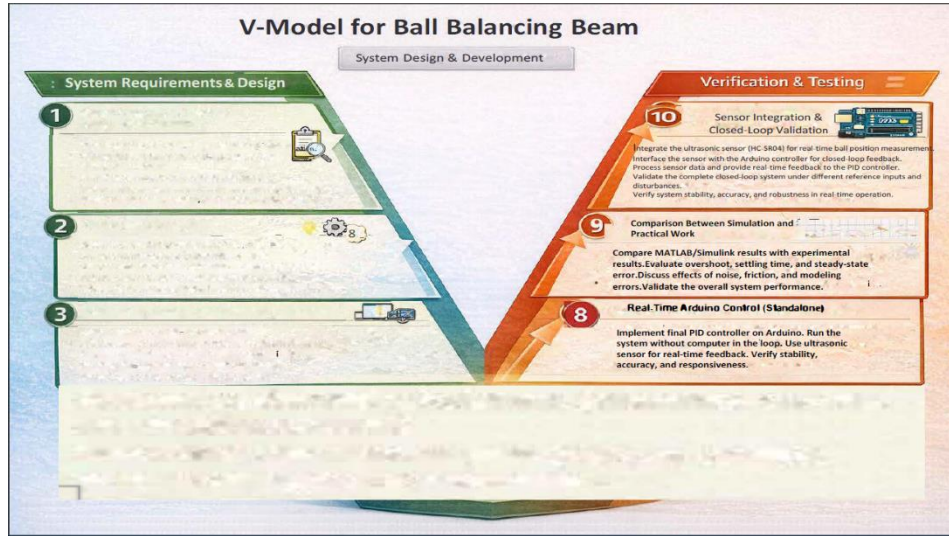


Figure 27: The right side of V-model

Conclusion

The results obtained show that the collected data contained a small level of system deflection, which had a noticeable effect on the motion of the ball. This deflection caused deviations from the ideal behavior of the system and directly influenced the accuracy of the response. Several attempts were made to compensate for this effect as much as possible, and through tuning and adjustments, the system was able to reach a relatively stable operating condition.

During the experimental process, a clear difference was observed between the theoretical calculations and the practical measurements. This difference was mainly due to the presence of various sources of noise in the system. Significant noise was introduced by the ultrasonic sensor, in addition to noise generated by the servo motor and other electronic components. These noise sources accumulated and had a strong impact on the overall system response, causing fluctuations, overshoot, and delays in the output signal.

Despite these challenges, it was concluded that the noise effects can be reduced and partially controlled by applying an appropriate PID controller. By tuning the PID parameters, the system response was improved, and a more acceptable level of stability and performance was achieved. Although the response was not perfectly ideal, it became sufficiently reliable for practical operation.

Finally, a comparison was made by plotting the system response for both the theoretical model and the practical implementation. The plots showed noticeable differences in terms of

overshoot, settling behavior, and overall dynamics. These differences confirm that real-world systems are always affected by non-ideal factors such as noise, disturbances, and hardware limitations. Nevertheless, the results demonstrate that with proper control techniques and tuning, the system dynamics can be adjusted to obtain a reasonably acceptable and functional response.

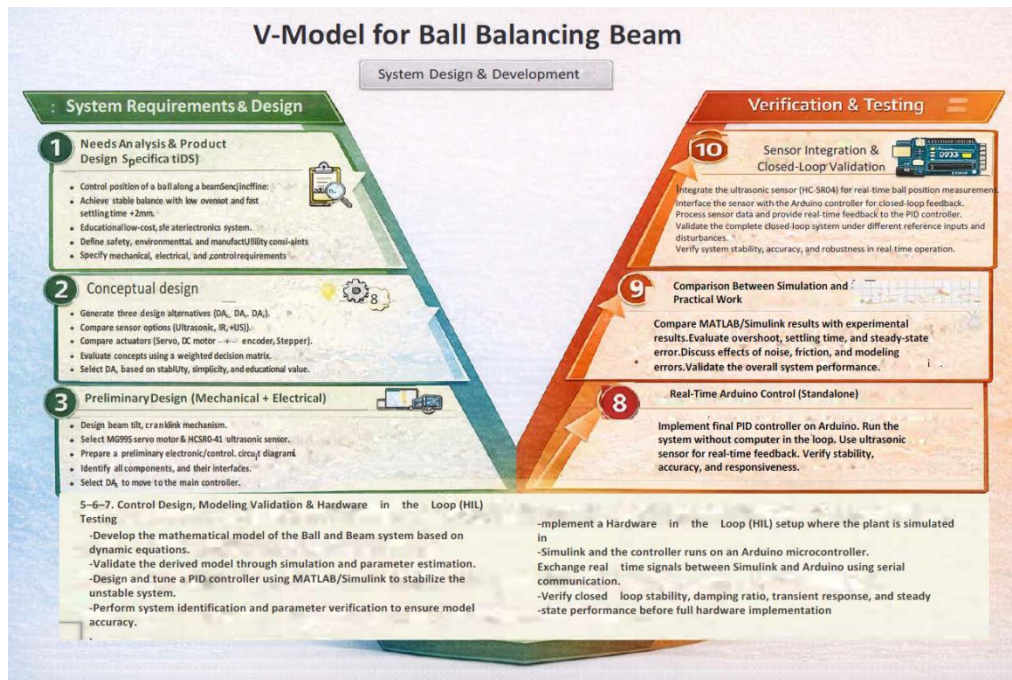


Figure 28: The V-model

References

- [1] Dr Amr Shafiq Lectures.
- [2] Eng Mohamed Ashraf Section
- [3] MathWorks, **MATLAB and Simulink Documentation**,
- [4] <https://www.mathworks.com/help>
- [5] MathWorks, **System Identification Toolbox User's Guide**,
- [6] <https://www.mathworks.com/products/sysid.html>
- [7] MG995 Servo Motor Datasheet, TowerPro <https://www.towerpro.com.tw>
- [8] Arduino Documentation, <https://www.arduino.cc/reference/en/> .
- [9] [Physical Modeling and Parameters Identification of the MG995 Servomotor paper](#)
- [10] <https://ctms.engin.umich.edu/CTMS/index.php?example=BallBeam§ion=SimulinkModelingn>